TCGA AML database documentation

Database Overview

Acute Myeloid Leukemia (AML) is a type of cancer in which the bone marrow uncontrollably produces an extensive amount of abnormal blood cells. Our database obtained AML data from The Cancer Genome Atlas (TCGA) project, specifically from the Firehose Legacy (GDAC Firehose) pipeline. The data were obtained from 200 patients; for each patient, the study collected various types of data, enabling patient-level omics and clinical discoveries. These data include gene expression profiles (RSEM), mutations, clinical records, and patient information.

After integrating these datasets into a relational database, we further enable NoSQL, such as Neo4j, to establish a connection between nucleotide-level mutations, patients, and their corresponding genes. This allows for the construction of SNP-gene-based relationships that reflect meaningful biological interactions.

Database Server

- Server: db via TCP/IP
- Server type: MySQL
- Server version: 9.5.0
- Protocol version: 10
- User: root@172.18.0.2
- Server charset: UTF-8 Unicode (utf8mb4)

Data Cleaning Decisions

Copy Number Alteration (CNA) and DNA methylation data were excluded to solely focus on the mutation and RNA expression relationship. Although this allows us to maximize the computational efficiency ans simply the data insertion process. Our database cannot handle expression change driven by alternation in copy number or promoter/enhancer methylation. It is likely for us to introduce bias in the downstream mutation expression-association analysis.

Despite CNA and DNA methylation, the TCGA dataset still comprises various data types, such as patient information, sample metadata, RSEM format mRNA-Seq, and mutation profiles. These datasets provide a robust background for analyzing the relationship between mutations and gene expression. As a result, a large proportion of these data was integrated into the database.

Although TCGA data provides comprehensive data types, some tables contain a large number of missing or unavailable values. In the patient dataset, the following data types were excluded because they were marked as “not available”: Karnofsky performance score, ECOG score, performance status timing, performance status other timing, method of initial sample procurement, disease code, ICD 10, ICD O3 Histology, ICD O3 site, project code, and primary site (other). Furthermore, to reduce the size of the database and improve computational efficiency, we removed administrative information such as completion date, molecular studies others performed, and informed consent verification status that were irrelevant to the scope of our study.

Sample information was also evaluated, and several data points were excluded due to missing or unavailable values. These excluded data types were specifically specimen current weight,

days to collection, days to specimen collection, specimen freezing method, sample initial weight, specimen second longest dimension, longest dimension, method of sample procurement, oct embedded, other method of sample procurement, pathology report file name, pathology report UUID, shortest dimension, time between clamping and freezing, and time between excision and freezing.

For the mutation and RSEM datasets, mutations that could not be mapped to the RSEM RNA expression data were removed, as they could not be interpreted in the gene expression analysis. In addition, variables in the mutations dataset that were completely unavailable were excluded; these variables include dbSNP val status.

Despite checking missing or unavailable values, we also verified whether the values in each column respected their intended annotation. During this process, we found that some entries in the HUGO symbol column contained date-like values. These incorrect entries were then marked as “N/A” to ensure a valid interpretation. Furthermore, we also found that the transcript names were written in a non-uniform format; therefore, and it was excluded to avoid inconsistencies and potential mismatches.

Lastly, we handled all missing and unavailable values, duplication, and format issues that occurred in the TCGA datasets. Missing values were retained as NULL, while the completely empty columns were excluded. Unexpectedly, we did not identify any duplicate rows across the datasets, suggesting that no deduplication procedure was needed. Lastly, we observed several formatting inconsistencies, which were resolved by standardizing the first letter of each word to uppercase and the remaining letters to lowercase.

Database Design & Table Relationship

Since we applied 4 TCGA datasets (patient, sample, mutation, and RSEM RNA expression data) to design this database, we further divided these original datasets into multiple retinol tables to satisfy the database normalization requirement.

The patient data were differentiated into the patient, patient_condition, treatment, and survival_record tables. The patient table stores basic demographic information such as patient ID, sex, race, and ethnicity. While the patient_condition table captures disease-related data, including clinical diagnostic results and medical histories. The treatment table stores treatment-related information, and the survival_record table contains outcomes of the treatment in terms of survival status and survival time. In this schema, we assume that an individual patient can be mapped to one record in the patient_condition, treatment, and survival_record tables, while one record in the patient_condition, treatment, and survival_record tables can be mapped to multiple patients, forming a one-to-many relationship. This design not only simplifies the database schema but also satisfies the normalization requirement.

The sample data were maintained in the cancer_sample and cancer_type tables. The cancer_type table describes the cancer classification and its characteristics, whereas the cancer_sample table contains the sample IDs and various related information. The cancer_sample table can be linked back to a single corresponding patient through a foreign key, forming a one-to-many relationship in which one patient can donate multiple samples. Similarly, the cancer_type table is connected ot the cancer_sample table, also forming a one-to-many relationship, where one cancer type can be associated with multiple samples.

The mutation data were divided into various tables, including mutation, allele, gene, transcript, consequence, and experiment. The mutation table serves as a central hub that connects mutation-related information through foreign key relationships. The mutation table itself consists of identifiers such as mutation_id, cancer_sample_id, gene_id, allele_id, consequence_id, and experiment_id, which link to cancer samples and various mutation characteristics. Furthermore, it also contains several genomic location information and mutation-specific annotations.

The experiment table captures sequencing-related data, such as tumor and sample barcodes, sequencing methods, center, and validation status. The allele table stores nucleotide-level changes, including both reference alleles and tumor alleles. The transcript table specifies which transcript the mutation occurred in. Lastly, a consequence table was compiled to describe the mutation's effects. These tables are all connected to the mutation tables through one-to-many relationships, where a single experiment, allele, consequence, or transcript can be associated with multiple mutations.

Finally, the RSEM RNA expression data were stored in the gene_expression table. This table serves as a connection between the gene and the cancer sample, which allows the database to analyze the gene expression together at the sample levels and compare the expression between different samples.

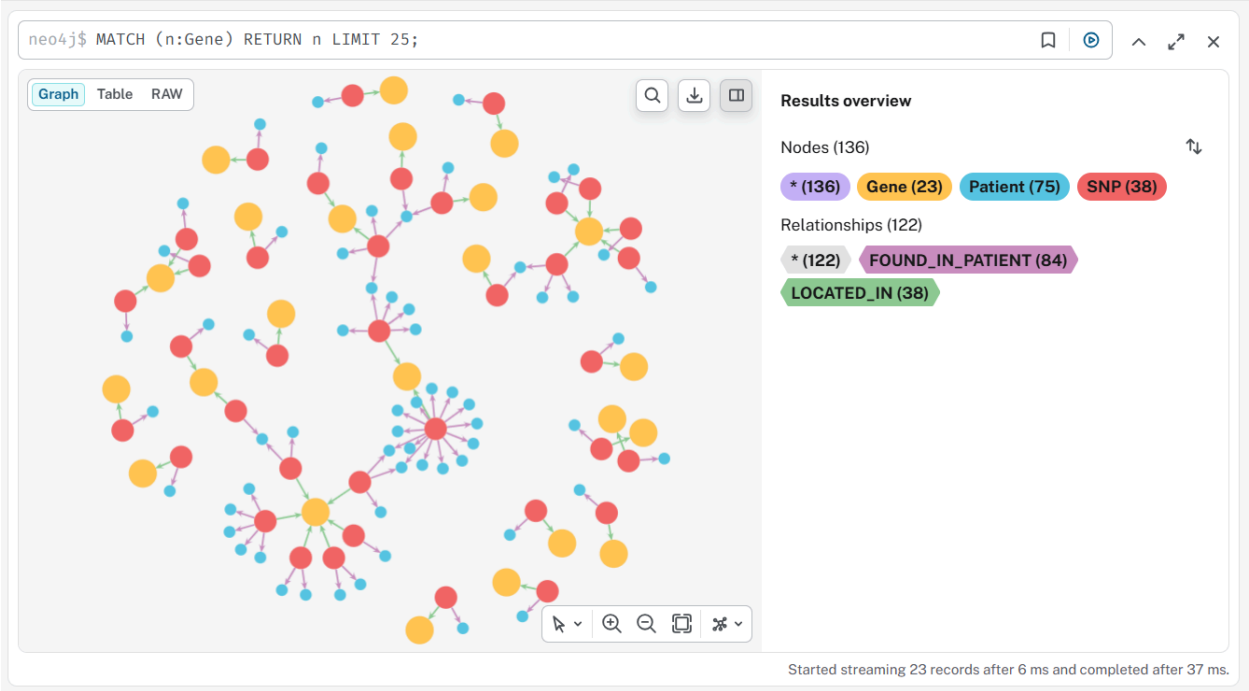
In addition to the SQL database, we also included a Neo4j graph database to visualize the relationship between SNP mutations, genes, and patients. The data from the Neo4j database was extracted from the MySQL relational database. Using the SQL script SQL_extract.txt, we extracted key characteristics, including dbSNP RS identifier, HUGO gene symbol, protein-level change, patient TCGA identifier, and consequence. These features allow us to track which SNPs are associated with which patients, which genes are affected, and what consequences the mutation causes, and vice versa.

The Neo4j database consists of four nodes and two relationship types. The nodes are patient, SNP, gene, and consequence. The patient node represents patients by their corresponding TCGA identifiers. The gene node represents genes by their corresponding HUGO symbols. The SNP node represents SNPs by their corresponding RS identifiers and also stores detailed mutation annotations, including amino acid changes, polarity, hydrophobicity, and more. This extra information helps us better interpret the potential impact of each SNP mutation. The consequence node represents the structural impact of a mutation on the transcript.

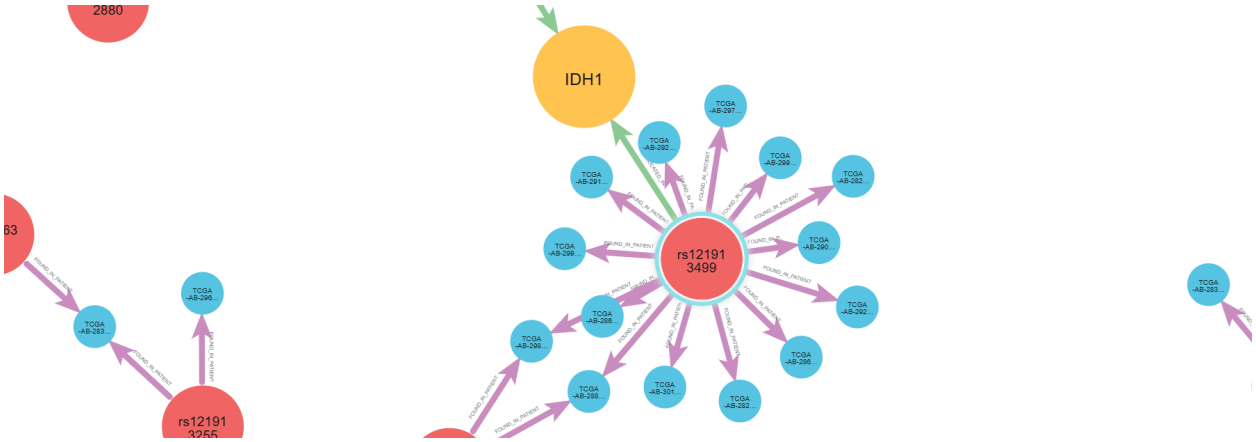
The relationship types in the Neo4j database are found_in_patient and located_in. The found_in_patient connects the SNP node to the patient node, clarifying that the SNP was detected in that patient. The located_in relationship connected the SNP node to a gene node, indicating that the SNP was located in that gene.

This graph database enables better visualization of the relationships between genes, SNPs, and patients. It also allows easier recognition of which SNP may be most critical for AML development by counting the number of patients associated with each SNP.

Neo4j database schema:



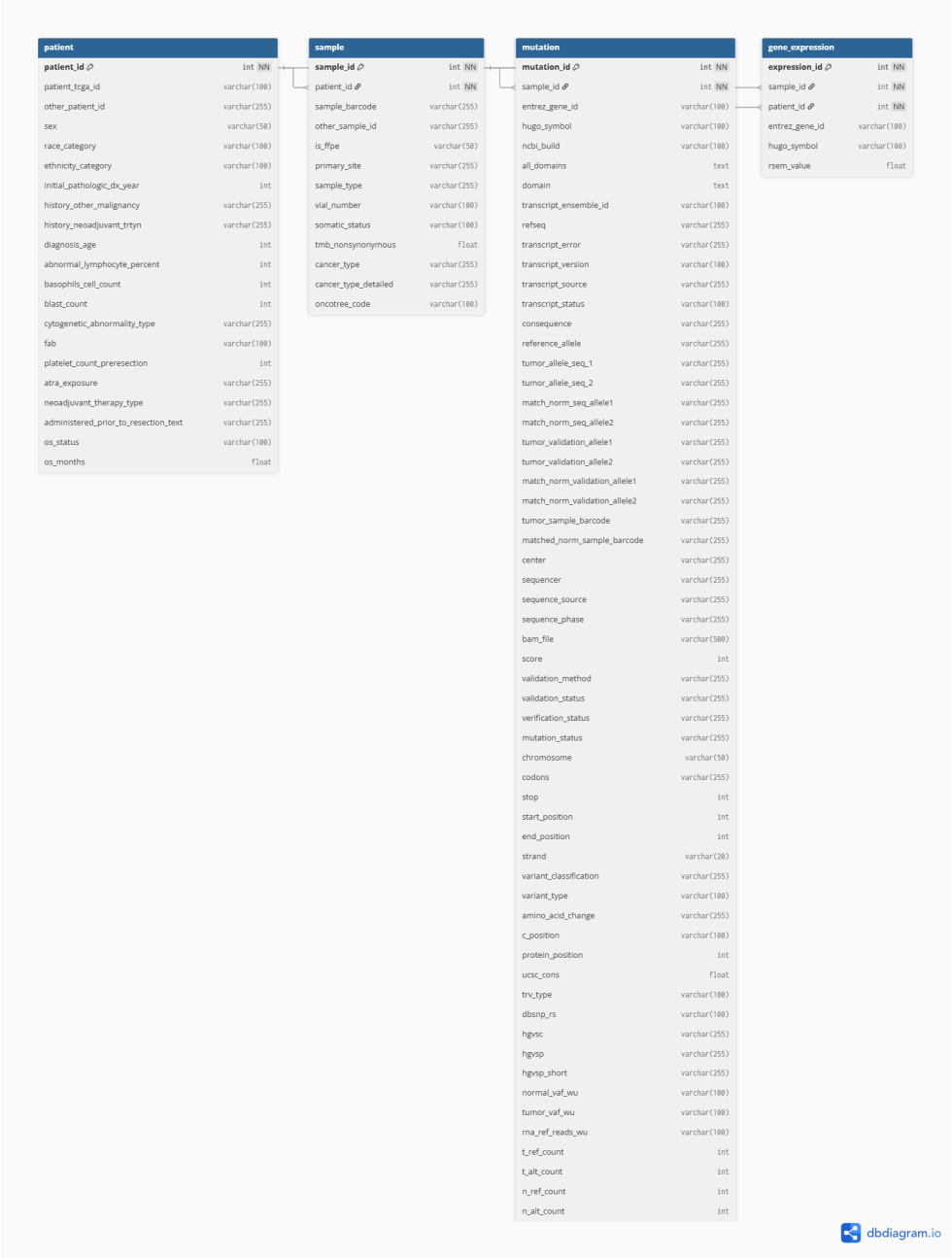
Central SNP that might be critical for AML development



Normal Forms and Deviations

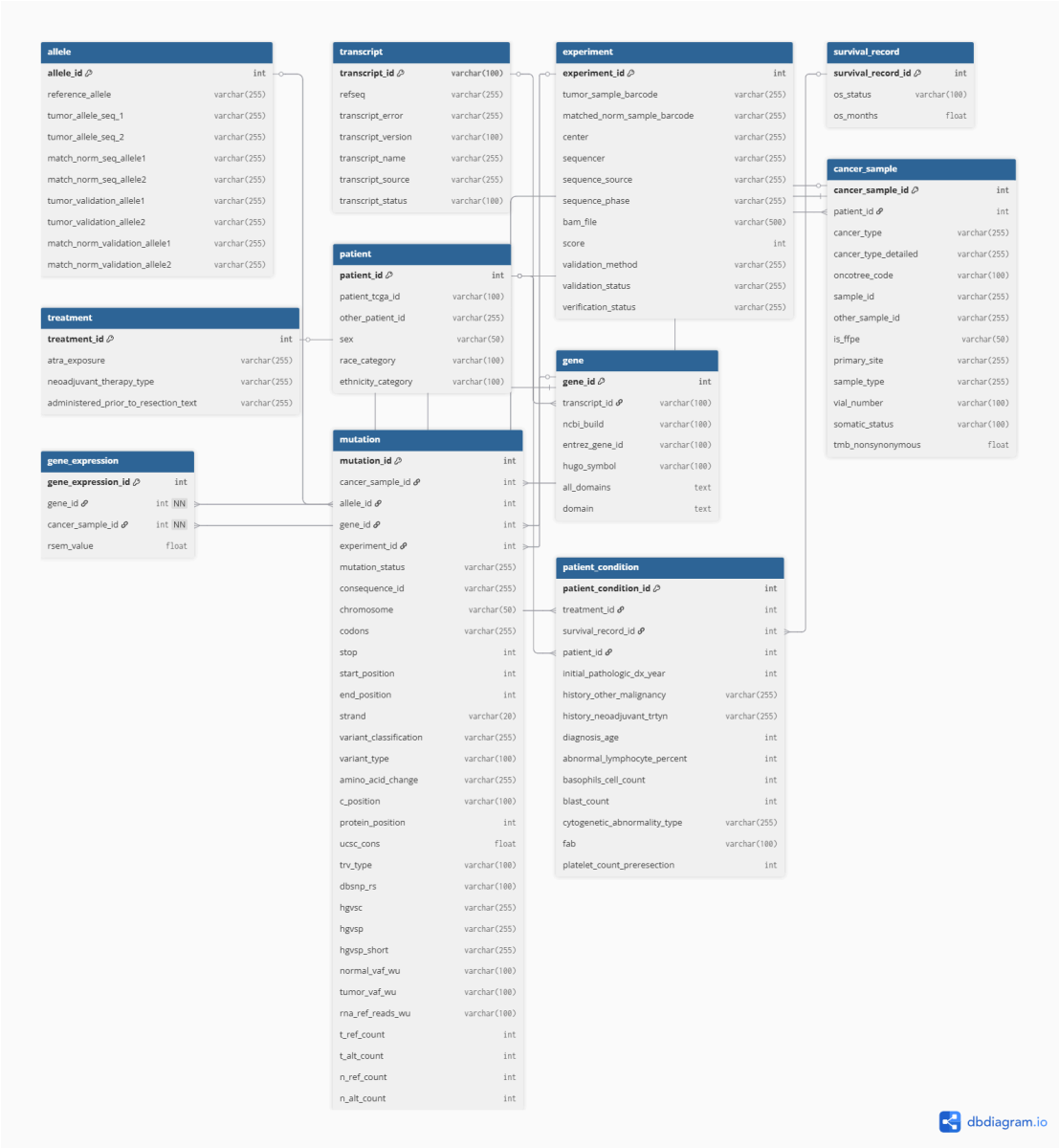
The database normalization rules follow a series of processes called normal forms, and the table must satisfy the requirements of all prior forms before proceeding. The first normal form (1NF) requires that there be no repeating groups or arrays. Our 1 NF consists of four tables: patient, cancer_samples, mutation, and gene expression, each derived from the raw TCGA datasets. During this stage, we ensure that each table contains only atomic values and define a primary key for identification. Lastly, a series of foreign keys was assigned to connect each table.

1NF diagram:



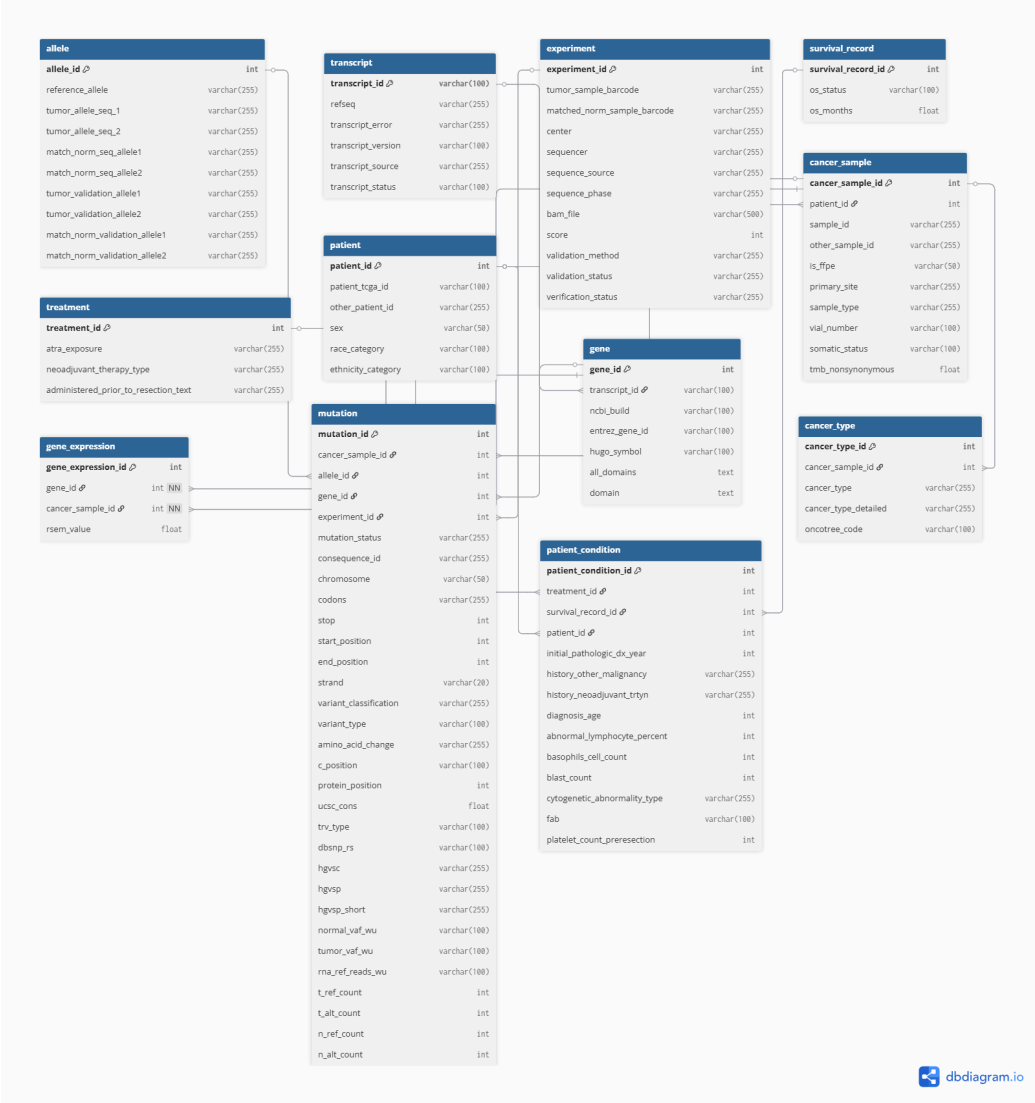
The second normal form (2NF) builds upon the 1NF by requiring that non-prime attributes depend on only part of a composite primary key. To satisfy the 2NF rules, attributes that did not depend on the primary key were separated into an isolated table. In this case, information such as allele sequence, transcripts, gene annotations, and experiments was extracted into the allele, transcript, gene, and experiment tables, respectively. Similarly, features that reflect the patient's clinical conditions, treatment, and survival records were removed into separate tables rather than being stored in a single table.

By separating the 1NF, we have now satisfied the rule of attribute-key dependency and eliminated data redundancy. With these adjustments, the schema is ready to proceed to the Third Normal Form (3NF).
2NF diagram:



The 3NF demands the elimination of transitive dependencies, meaning that non-prime attributes should not depend on other non-prime attributes, but only the primary key. In our case, the cancer_sample table generated at 2NF contained attributes that were indirectly dependent on the primary key cancer_sample_id, since every sample in this database belongs to the AML cohort, cancer type-related information, such as cancer_type, cancer_type_detailed, and oncotree_code, was repeated across many samples. To resolve this issue, these attributes were removed from the cancer_sample table and placed into a separate cancer_type table. The

cancer_sample now stores only sample-specific information, while the new cancer_type stores cancer identity information.
This 3NF also satisfies Boyce–Codd Normal Form (BCNF) because all functional dependencies in each table have a determinant candidate key. In our schema, each table is defined by a single primary key, and all non-key attributes depend on their corresponding keys. There were no cases where the non-key attributes were determined by another non-key attribute.
3NF diagram:



Moving on to the Fourth Normal Form (4NF), the original 3NF schema does not need to be altered because it already avoids multivalued dependencies. 4NF requires the schema to be in BCNF and have no two or more independent multi-valued attributes that are not directly related to each other. In our case, the potential multi-valued attributes, such as mutations per sample and gene expression per sample, were already separated into independent tables.
Finally, the Fifth Normal Form (5NF) requires the elimination of the join dependencies, meaning eliminating redundant data by decomposing tables into a smaller size that doesn't lose any information. In our case, the consequence attribute was removed from the mutation table and

placed into an isolated consequence table. As multiple consequences can be associated with a mutation, creating a potential joint dependency. By separating the consequence into its own table, we connect the consequence table with the mutation table through a foreign key. This design ensures a lossless decomposition for the mutation table, as the original relationship can be reconstructed, thereby satisfying the requirements of 5NF.
5NF schema:

Data Dictionary & Table Relationship

Each table within the database is documented below. Where each table consists of its corresponding attributes, primary keys, foreign keys, data types, and a brief description of the role of keys and attributes.

Data dictionary for the *allele* table

Attribute	Type	Key	Definition & Description
allele_id	int	primary	A unique identifier assigned to each

		key	allele record
Reference_allele	varchar(255)	N/A	The reference allele sequence at the mutation position
Tumor_allele_seq_1	varchar(255)	N/A	The first allele sequence detected in the tumor sample
Tumor_allele_seq_2	varchar(255)	N/A	The second allele sequence detected in the tumor sample
match_norm_seq_allele1	varchar(255)	N/A	The first allele sequence that matched the normal sample
match_norm_seq_allele2	varchar(255)	N/A	The second allele sequence that matched the normal sample
tumor_validation_allele1	varchar(255)	N/A	The first validated allele sequence confirmed by tumor validation
tumor_validation_allele2	varchar(255)	N/A	The second validated allele sequence was confirmed by tumor validation
match_norm_validation_allele1	varchar(255)	N/A	The first validated allele sequence identified in the matched normal sample
match_norm_validation_allele2	varchar(255)	N/A	The second validated allele sequence identified in the matched normal sample

Data dictionary for the *cancer_sample* table

Attribute	Type	Key	Definition & Description
cancer_sample_id	int	primary key	A unique identifier assigned to each cancer sample record
patient_id	int	foreign key	The foreign key that linked each samcer sample to its corresponding patient
sample_id	varchar(255)	N/A	TCGA format identifier that represents a specific sample
other_sample_id	varchar(255)	N/A	Additional identifier associated with the sample
is_ffpe	varchar(50)	N/A	Indicates whether the sample is prepared using the Formalin-Fixed Paraffin-Embedded (FFPE) method
primary_site	varchar(255)	N/A	The primary location from which the tumor sample was obtained

sample_type	varchar(255)	N/A	The biological classification of the tumor sample
vial_number	varchar(255)	N/A	The vial number associated with the stored sample
somatic_status	varchar(255)	N/A	Indicates whether the mutation is classified as somatic or germline
tmb_nonsynonymous	float	N/A	The number of nonsynonymous mutations per megabase detected in the sample

Data dictionary for the *cancer_type* table

Attribute	Type	Key	Definition & Description
cancer_type_id	int	primary key	A unique identifier assigned to each cancer type record
cancer_sample_id	int	foreign key	Foreign key linking the cancer type to the corresponding cancer sample
cancer_type	varchar(255)	N/A	A classification of the cancer type associated with the sample
cancer_type_detailed	varchar(255)	N/A	A detailed description of the cancer classification
oncotree_code	varchar(100)	N/A	The OncoTree classification code was used to represent the cancer type

Data dictionary for the *consequence* table

Attribute	Type	Key	Definition & Description
consequence_id	int	primary key	A unique identifier assigned to each consequence record
consequence	varchar(255)	N/A	A description of the effect of a mutation on a protein

Data dictionary for the *experiment* table

Attribute	Type	Key	Definition & Description
experiment_id	int	primary key	A unique identifier assigned to each experiment record
tumor_sample_barco	varchar(255)	foreign	TCGA barcode used for tumor sample

de		key	identification during the experiment
matched_norm_sample_barcode	varchar(255)	N/A	TCGA barcode used to identify the matched normal sample corresponding to the tumor sample
center	varchar(255)	N/A	The research center responsible for the experiment
sequencer	varchar(255)	N/A	The sequencing platform used to generate the experiment data
sequence_source	varchar(255)	N/A	Indicates the sequencing technique used for the experiment
sequence_phase	varchar(255)	N/A	Indicates the sequencing phase associated with the experiment
bam_file	varchar(255)	N/A	Identifier or reference associated with the BAM file
score	int	N/A	Numerical quality score of the experiment
validation_method	varchar(255)	N/A	Experimental method used to validate detected variants
validation_status	varchar(255)	N/A	Indicates the validation status of the detected variant
verification_status	varchar(255)	N/A	Indicates verification of the status of the experiment results

Data dictionary for the *gene* table

Attribute	Type	Key	Definition & Description
gene_id	int	primary key	A unique identifier assigned to each gene record
transcript_id	int	foreign key	Foreign key linking the gene to its corresponding transcripts
ncbi_bulid	varchar(100)	N/A	Reference genome assembly version used for annotation
entrez_gene_id	varchar(100)	N/A	Entrez Gene Identifier assigned by NCBI to represent a specific gene
hugo_symbol	varchar(100)	N/A	Gene symbol assigned by the HUGO Gene Nomenclature Committee

all_domains	text	N/A	A list of all known protein domains associated with the gene
domain	text	N/A	Specific protein domain associated with the gene

Data dictionary for the *gene_expression* table

Attribute	Type	Key	Definition & Description
gene_expression_id	int	primary key	A unique identifier assigned to each gene expression record
gene_id	int	foreign key	Foreign key linking the expression record to the corresponding gene
cancer_sample_id	int	foreign key	Foreign key linking the expression record to the corresponding sample
rsem_value	float	N/A	Gene expression value quantified using the RSEM method

Data dictionary for the *mutation* table

Attribute	Type	Key	Definition & Description
mutation_id	int	primary key	A unique identifier assigned to each mutation record
cancer_sample_id	int	foreign key	Foreign key linking the mutation record to the corresponding cancer sample
allele_id	int	foreign key	Foreign key linking the mutation record to the corresponding allele
gene_id	int	foreign key	Foreign key linking the mutation record to the corresponding gene
consequence_id	int	foreign key	Foreign key linking the mutation record to the corresponding consequence
experiment_id	int	foreign key	Foreign key linking the mutation record to the corresponding experiment
mutation_status	varchar(255)	N/A	Indicates whether the mutation is classified as somatic or germline
chromosome	varchar(50)	N/A	Chromosome on which the mutation is located
codons	varchar(255)	N/A	Codon sequence change caused by the

			mutation
stop	int	N/A	Genomic start position of the mutation
start_position	int	N/A	Genomic start position of the mutation
end_position	int	N/A	Genomic end position of the mutation
strand	varchar(20)	N/A	The DNA strand where the mutation occurred
variant_classification	varchar(255)	N/A	Functional classification describes the impact of the mutation, which includes missense, nonsense, and silent mutations
variant_type	varchar(100)	N/A	Describes the structural type of the mutation
amino_acid_change	varchar(255)	N/A	Describes the amino acid change caused by the mutation
c_position	varchar(100)	N/A	Coding DNA position where the mutation occurred
protein_position	int	N/A	The amino acid position where the mutation occurred within the encoded protein
ucsc_cons	float	N/A	UCSC conservation score representing the evolutionary conservation of the mutation
trv_type	varchar(100)	N/A	Transcript variant type associated with the mutation
dbsnp_rs	varchar(100)	N/A	dbSNP RS identifier associated with the mutation
hgvsc	varchar(255)	N/A	HGVS type of annotation describing a mutation at the DNA sequence level
hgvsp	varchar(255)	N/A	HGVS type of annotation describing a mutation at the protein level
hgvsp_short	varchar(255)	N/A	Shortened HGVS protein-level annotation
normal_vaf_wu	varchar(100)	N/A	The variant allele frequency observed in

			the normal sample generated by the Washington University pipeline
tumor_vaf_wu	varchar(100)	N/A	The variant allele frequency observed in the tumor sample generated by the Washington University pipeline
rna_ref_reads_wu	varchar(100)	N/A	Number of reads that mapped to the reference genome generated by the Washington University pipeline
t_ref_count	int	N/A	Number of reads supporting the reference allele in the tumor sample
t_alt_count	int	N/A	Number of reads supporting the alternative allele in the tumor sample
n_ref_count	int	N/A	Number of reads supporting the reference allele in the normal sample
n_alt_count	int	N/A	Number of reads supporting the alternative allele in the normal sample

Data dictionary for the *patient* table

Attribute	Type	Key	Definition & Description
patient_id	int	primary key	A unique identifier assigned to each patient record
patient_tcga_id	varchar(100)	N/A	TCGA patient identifier
other_patient_id	varchar(255)	N/A	Secondary patient identifier
sex	varchar(50)	N/A	Biological sex of the patient
race_category	varchar(100)	N/A	Racial classification of the patient
ethnicity_category	varchar(100)	N/A	Ethnic classification of the patient

Data dictionary for the *patient_condition* table

Attribute	Type	Key	Definition & Description

patient_condition_id	int	primary key	A unique identifier assigned to each patient record
treatment_id	int	foreign key	Foreign key linking the patient condition record to the corresponding treatment type
survival_record_id	int	foreign key	Foreign key linking the patient condition record to the corresponding survival record
patient_id	int	foreign key	Foreign key linking the patient condition record to the corresponding patient
initial_pathologic_dx_year	int	N/A	Date on which the patient received the initial diagnosis
history_other_malignancy	varchar(255)	N/A	Patient's history of other malignant diseases
history_neoadjuvant_trtyn	varchar(255)	N/A	Patient's history of neoajuvant treatment before the current treatment
diagnosis_age	int	N/A	Age of the patient at the time of diagnosis
abnormal_lymphocyte_percent	int	N/A	Percentage of abnormal lymphocytes detected in the sample
basophils_cell_count	int	N/A	The count of basophil cells inside the patient sample
blast_count	int	N/A	The count of blast cells inside the patient sample
cytogenetic_abnormality_type	varchar(255)	N/A	Type of cytogenetic abnormalities identified in the patient
fab	varchar(100)	N/A	The French-American-British (FAB) classification assigned to leukemia
platelet_count_preresection	int	N/A	The count of platelets inside the patient sample before surgery

Data dictionary for the *survival_record* table

Attribute	Type	Key	Definition & Description
survial_record_id	int	primary key	A unique identifier assigned to each survival record
os_status	varchar(100)	N/A	Overall survival status of the patient
os_months	float	N/A	Overall survival duration of the patient

Data dictionary for the *transcript* table

Attribute	Type	Key	Definition & Description
transcript_id	int	primary key	A unique identifier assigned to each survival record
transcript_ensemble_id	varchar(100)	N/A	Overall survival status of the patient
refseq	varchar(255)	N/A	Overall survival duration of the patient
transcript_error	varchar(255)	N/A	Description of any observed issues regarding the transcript record
transcript_version	varchar(100)	N/A	Transcript annotation version
transcript_source	varchar(255)	N/A	Transcript information provider
transcript_status	varchar(100)	N/A	Annotation status of the transcript

Data dictionary for the *treatment* table

Attribute	Type	Key	Definition & Description
treatment_id	int	primary key	A unique identifier assigned to each treatment record
atra_exposure	varchar(255)	N/A	Describe whether the patient was exposed to all-trans retinoic acid (ATRA)

neoadjuvant_therapy_type	varchar(255)	N/A	List the neoadjuvant therapy assigned to the patient before the current treatment
administered_prior_to_resection_text	varchar(255)	N/A	A description indicating the treatments assigned to the patient before surgery

By reviewing the data dictionary listed above, the tables patient, mutation, cancer_sample, and gene_expression served as the hub tables of the TCGA AML database. These tables were directly derived from the row TCGA datasets and connect the remaining tables through foreign keys. The patient table stores the patient's demographic information, and it's linked to the patient_condition table and cancer_sample table through the foreign key patient_id. The cancer_sample table is linked to the mutation, gene_expression, and cancer_type tables. The mutation table is acting as a central hub that links the mutation record to the allele, gene, consequence, experiment, and cancer_sample tables. Lastly, the gene_expression table acted as a bridge connecting genes and samples, enabling gene expression analysis at the sample level.

Script Map & Reconstruction Instructions

The TCGA AML database was constructed using a virtual machine created in the Oracle VM VirtualBox. The virtual machine runs on a Linux Fedora operating system, which serves as an environment for database operation and management. The system was configured with 4096 of RAM and 25 GB of virtual storage. Within the virtual machine, the database infrastructure was executed in a MySQL community server (Version 9.5.0). While database administration was performed in phpMyAdmin (Version 5.2.3), which can be accessed locally through a browser (localhost:8888 / db | phpMyAdmin 5.2.3).

To reconstruct the database, first import the virtual machine image into Oracle VM VirtualBox and ensure that Apache and MySQL are running properly. Once the virtual machine is activated, open a browser locally and navigate to the phpMyAdmin interface. Inside phpMyAdmin, create a new database and named as TCGA_AML_db. After creating the database, import the SQL schema script that defines the structure of the database (DDL_TCGA_AML_db.txt).

The cleaned datasets were converted into Data Manipulation Language (DML) previously, and should then be imported in a dependency order to establish a relationship. Tables without foreign keys should be imported first; these tables include patient (SQL_insert_patient.txt), transcript (SQL_insert_transcript.txt), consequence (SQL_insert_consequence.txt), allele (SQL_insert_allele.txt), treatment (SQL_insert_treatment.txt), and experiment (SQL_insert_experiment.txt). Tables containing foreign keys should be imported afterward, including gene (SQL_insert_gene.txt), cancer_sample (SQL_insert_cancer_sample.txt), cancer_type (SQL_insert_cancer_type.txt), patient_condition (SQL_insert_patient_condition.txt), gene_expression (SQL_insert_gene_expression.txt), survival_record (SQL_insert_survival_record.txt), and mutation (SQL_insert_gene_mutation.txt).

After importing all DML scripts, the database can be examined by executing test SQL queries to ensure the correct relationships between tables were established.

In addition to the MySQL database, we also generated a Neo4j graph database to visualize the relationship between genes, SNPs, and patients. The Neo4j database was constructed in a

local Neo4j instance using a local web browser. The instance was connected through bolt://localhost:7687, with the user set to neo4j and Neo4j community version 2025.11.2.
To reconstruct the Neo4j database, first open the MySQL database and extract information such as genes, patients, SNPs, and consequences using the SQL stored in script SQL_extract.txt. Next, run fetch_dbSNP_annotations.py to obtain SNP annotations from the NCBI database. These extra annotations help us better interpret the potential biological impact of each SNP.
After preparing all data files, apply the Python code generate_neo4j_cypher.py to create three Cypher scripts: neo4j_import.cypher, neo4j_chemistry.cypher, and dbsnp_annotations.cypher. Executed these scripts in the Neo4j browser in the respective order. After all scripts are executed, the Neo4j database will be fully constructed.

Limitations and future work

A major limitation of this study is the exclusion of copy number alteration (CNA), RPKM RNA-seq data, and DNA methylation data to improve computational efficiency. This exclusion restricts our ability to fully interpret the relationship between mutations and gene expression. In real-world biology, changes in gene expression are not only driven by nucleotide-level mutations, but they can also be altered by epigenetic changes such as methylation at promoter or enhancer regions, which may silence the influence of sequence-level changes.
Similarly, the CNA data allows us to access how alternation at the DNA segment level affects gene amplifications and deletions. The absence of CNA data makes it more difficult to determine whether changes in expression are driven by mutations or variations in copy number.
Although we incorporated RSEM RNA-seq data for expression analysis, the missing RPKM data provided an extra normalized RNA expression measurement, which can make the expression value more interpretable and comparable across different genes. This limits our ability to cross-compare expression patterns and further reduces the interpretability of our results.
Further work can incorporate additional information, such as CNA, RPKM-based RNA-seq data, and DNA methylation data, to provide a more comprehensive result on gene regulation in AML. The integration of these data can provide a more accurate interpretation of the relationship between gene expression and mutations. Furthermore, the expansion of the database can potentially enable the discovery of tumor biomarkers and therapeutic targets.